CAT 1
REFERENCE MATERIAL -I
COMPUTER NETWORKS

1. ping

- **Purpose**: Tests connectivity between your computer and a remote device.
- **Example**: ping google.com

2. ipconfig (Windows) / ifconfig (Linux/Mac)

- **Purpose**: Displays or configures the IP configuration of a device.
- **Example**: ipconfig /all (Windows), ifconfig (Linux/Mac)

3. tracert (Windows) / traceroute (Linux/Mac)

- **Purpose**: Displays the route packets take to reach a destination.
- **Example**: tracert google.com (Windows), traceroute google.com (Linux/Mac)

4. netstat

- **Purpose**: Displays network statistics and active connections.
- **Example**: netstat -an (Lists all active connections)

5. nslookup

- **Purpose**: Queries DNS to obtain domain name or IP address mapping.
- **Example**: nslookup google.com

6. dig (Linux/Mac)

- **Purpose**: Retrieves DNS information, similar to nslookup but more detailed.
- **Example**: dig google.com

7. arp

- **Purpose**: Displays or modifies the ARP (Address Resolution Protocol) cache.
- **Example**: arp -a (Shows all ARP entries)

8. route

- **Purpose**: Displays or modifies the IP routing table.
- **Example**: route print (Displays the current routing table)

9. hostname

- **Purpose**: Displays the hostname of the machine.
- **Example**: hostname

10. whois

- **Purpose**: Queries databases to find information about domain names or IP addresses.
- **Example**: whois google.com

11. telnet

- **Purpose**: Allows you to connect to remote systems over TCP/IP, often used for testing ports.
- **Example**: telnet example.com 80

12. ftp

- **Purpose**: Transfers files between computers over a network using the FTP protocol.
- **Example**: ftp ftp.example.com

13. scp

- **Purpose**: Securely copies files between hosts over a network.
- **Example**: scp file.txt user@remote:/path/to/destination

14. curl

- **Purpose**: Transfers data from or to a server, often used to test URLs and APIs.
- **Example**: curl http://example.com

15. nmap

- **Purpose**: Scans a network to discover hosts, services, and open ports.
- **Example**: nmap 192.168.1.1

16. ssh

- **Purpose**: Securely connects to a remote device or server over the network.
- **Example**: ssh user@remote-host

17. tcpdump (Linux/Mac)

- **Purpose**: Captures and analyzes network traffic.
- **Example**: tcpdump -i eth0 (Captures traffic on the eth0 interface)

18. netsh (Windows)

- **Purpose**: Configures and displays various network settings and configurations.
- **Example**: netsh wlan show profiles (Displays Wi-Fi profiles on a Windows machine)

19. iwconfig (Linux)

- **Purpose**: Configures wireless network interfaces.
- **Example**: iwconfig wlan0 (Displays wireless configuration for wlan0)

20. nmcli (Linux)

- **Purpose**: Command-line tool for managing NetworkManager on Linux systems.
- **Example**: nmcli device status (Shows the status of network devices)

21. ip (Linux)

- **Purpose**: Displays and manipulates routing, devices, policy routing, and tunnels.
- **Example**: ip addr show (Displays IP addresses and network interfaces)

CAT 1
REFERENCE MATERIAL -I
COMPUTER NETWORKS

22. ss (Linux)

- **Purpose**: Displays detailed network socket information.
- **Example**: ss -tuln (Lists open ports)

23. mtr (Linux/Mac)

- **Purpose**: Combines the functionality of ping and traceroute, providing a continuous network diagnostics tool.
- **Example**: mtr google.com

24. tshark

- **Purpose**: A terminal-based version of Wireshark for capturing and analyzing network traffic.
- **Example**: tshark -i eth0 (Captures traffic on the eth0 interface)

25. wget

- **Purpose**: Downloads files from the web through HTTP, HTTPS, or FTP protocols.
- **Example**: wget http://example.com/file.zip

26. ethtool (Linux)

- **Purpose**: Displays and changes Ethernet device settings.
- **Example**: ethtool eth0 (Shows information about the eth0 interface)

27. nc (Netcat)

- **Purpose**: Reads and writes data across network connections using TCP or UDP.
- **Example**: nc -zv 192.168.1.1 80 (Checks if port 80 is open on 192.168.1.1)

28. fping

- **Purpose**: Sends ICMP echo requests to multiple hosts at once, useful for network discovery.
- **Example**: fping -a -g 192.168.1.0/24 (Pings all devices in the 192.168.1.0/24 network)

29. route

- **Purpose**: Displays and manipulates the routing table.
- **Example**: route add default gw 192.168.1.1 (Adds a default gateway)

30. nsenter (Linux)

- **Purpose**: Allows you to enter the namespaces of another running process, useful for network namespaces.
- **Example**: nsenter --net --target <pid> (Enters the network namespace of a process)

1. Parity bit

Parity bits and checksums are methods of detecting errors during transmission to ensure that data is not lost or corrupted. In this section, we will discuss both methods one by one.

CAT 1
REFERENCE MATERIAL -I
COMPUTER NETWORKS

Parity Bit: In the Parity bit method, an additional bit is added to each word to make error detection possible on the communication channel. The parity bit has two values, either 0 or 1. Even Parity and odd parity are types of parity.

- **Even Parity:** The sender is sending the message as a stream of bits over the communication channel. After adding a parity bit (1 or 0) to the message, the number of 1's becomes even then it is known as even-parity.
- **Odd Parity:** The sender is sending the message as a stream of bits over the communication channel. After adding a parity bit (1 or 0) to the message, the number of 1's becomes odd then it is known as odd-parity.

Example code:

```
#include <stdio.h>
// Function to calculate even parity for a byte
int calculateParity(int data[], int size) {
int parity = 0;
for (int i = 0; i < size; i++) {
parity ^= data[i];
}
return parity;
}
int main() {
int data[8];
int size = 8;
// Input 8-bit data from the user
printf("Enter 8-bit data (1 or 0 for each bit):\n");
for (int i = 0; i < size; i++) {
printf("Bit %d: ", i + 1);
scanf("%d", &data[i]);
}
// Calculate parity bit (for even parity)
int parity = calculateParity(data, size);
printf("Calculated Parity Bit (Even Parity): %d\n", parity); // Append the parity bit to the data for transmission
int transmittedData[9];
for (int i = 0; i < size; i++) {
transmittedData[i] = data[i];
}
transmittedData[size] = parity;
// Simulate reception and check parity at the receiver's end
int receivedParity = calculateParity(transmittedData, size + 1);
if (receivedParity == 0) {
printf("No error detected. Data received correctly.\n");
} else {
printf("Error detected in the received data.\n");
}
return 0;
}
```

Output

```
Enter 8-bit data (1 or 0 for each bit):
Bit 1: 1
Bit 2: 0
Bit 3: 1
Bit 4: 1
Bit 5: 0
```

Bit 6: 0
Bit 7: 1
Bit 8: 0
Calculated Parity Bit (Even Parity): 0
No error detected. Data received correctly.

1. Check sum

The problem with the parity bit method is that it reliably detects only one-bit errors in the message. If the message contains a burst error, the parity bit method will not work. Therefore, the parity bit is only used when cables are challenging for error detection, as those cables have very low error rates. To solve the problem of the parity bit method, the checksum method is used.

- The checksum is the complement of the total of all the code-words. The Checksum is placed at the end of the message. The checksum method operates on words instead of bits.

Example

```
#include <stdio.h>
// Function to calculate 1's complement sum of two 8-bit numbers
unsigned char onesComplementSum(unsigned char a, unsigned char b) {
unsigned short sum = a + b;
if (sum > 0xFF) { // Check for carry
sum = (sum & 0xFF) + 1;
}
return (unsigned char)sum;
}
// Function to calculate the checksum
unsigned char calculateChecksum(unsigned char data[], int size) {
unsigned char checksum = data[0];
for (int i = 1; i < size; i++) {
checksum = onesComplementSum(checksum, data[i]);
}
return ~checksum; // Return the complement of the sum }
// Function to validate the checksum at the receiver's end
int validateChecksum(unsigned char data[], int size, unsigned char receivedChecksum) {
unsigned char sum = data[0];
for (int i = 1; i < size; i++) {
sum = onesComplementSum(sum, data[i]);
}
sum = onesComplementSum(sum, receivedChecksum);
return sum == 0xFF; // If sum is 0xFF, no error detected
}
// Function to convert binary string to unsigned char
unsigned char binaryStringToChar(char *binaryStr) {
unsigned char value = 0;
for (int i = 0; i < 8; i++) {
value = (value << 1) | (binaryStr[i] - '0');
}
return value;
}
// Function to print an 8-bit number in binary format
void printBinary(unsigned char value) {
for (int i = 7; i >= 0; i--) {
printf("%d", (value >> i) & 1);
}
printf("\n");
}
```

```
int main() {
unsigned char data[4];
int size = 4;
char binaryStr[9]; // To store 8-bit binary string input
// Input 4 frames of 8-bit data from the user in binary
printf("Enter 4 frames of 8-bit data (in binary, e.g., 11001100):\n");
for (int i = 0; i < size; i++) {
printf("Frame %d: ", i + 1);
scanf("%8s", binaryStr);
data[i] = binaryStringToChar(binaryStr);
}
// Calculate the checksum at the sender's side
unsigned char checksum = calculateChecksum(data, size);
printf("Calculated Checksum (in binary): ");
printBinary(checksum);
// Simulate sending and receiving the data along with checksum
printf("\nSimulating reception of data...\n");
// Output received data frames and checksum
printf("Received Data Frames:\n");
for (int i = 0; i < size; i++) {
printBinary(data[i]);
}
printf("Received Checksum: ");
printBinary(checksum);
// Validate the checksum at the receiver's end
if (validateChecksum(data, size, checksum)) {
printf("No error detected. Data received correctly.\n");
} else {
printf("Error detected in the received data.\n");
}
return 0;
}
```

```
Enter 4 frames of 8-bit data (in binary, e.g., 11001100):
Frame 1: 11001100
Frame 2: 10101010
Frame 3: 11110000
Frame 4: 11000011
Calculated Checksum (in binary): 11010011
Simulating reception of data...
Received Data Frames:
11001100
10101010
11110000
11000011
Received Checksum: 11010011
No error detected. Data received correctly.
```

With error inputs

Frame 1: 11001100 Frame 2: 10101010 Frame 3: 11110000 Frame 4: 11000011

Will get error

Error Correction

The error detection method is used to check whether an error has occurred in the data. Now it's time to learn how we can fix the detected errors using error correction methods. The error correction method is used to find out the exact number of corrupted bits to correct.

Error correction can be done in two ways:

1 Backward Error Correction

2 Forward Error Correction

3 **Backward Error Correction:** In simple words, the receiver sends an acknowledgment to the sender when corrupt data is received, and the sender retransmits the data to the receiver.

Forward Error Correction: When the receiver receives data that contains errors, the receiver attempts to retrieve and correct the data using methods. The 4 main methods of Forward-Error correction are as follows: Hamming Codes

Binary Convolution Codes

Reed-Solomon Codes

Low-Density Parity-Check Codes

Hamming codes

Example Code:

```
#include <stdio.h>
// Function to calculate the parity bits
void calculateHamming(int data[], int hammingCode[]) {
// Data bits
hammingCode[2] = data[0];
hammingCode[4] = data[1];
hammingCode[5] = data[2];
hammingCode[6] = data[3];
// Parity bits
hammingCode[0] = hammingCode[2] ^ hammingCode[4] ^ hammingCode[6];
hammingCode[1] = hammingCode[2] ^ hammingCode[5] ^ hammingCode[6];
hammingCode[3] = hammingCode[4] ^ hammingCode[5] ^ hammingCode[6];
}
// Function to check and correct errors
int checkHamming(int hammingCode[]) {
int c1 = hammingCode[0] ^ hammingCode[2] ^ hammingCode[4] ^ hammingCode[6];
int c2 = hammingCode[1] ^ hammingCode[2] ^ hammingCode[5] ^ hammingCode[6];
int c4 = hammingCode[3] ^ hammingCode[4] ^ hammingCode[5] ^ hammingCode[6];
int errorPos = c1 + (c2 * 2) + (c4 * 4);
return errorPos;
}
int main() {
int data[] = {1, 0, 1, 1}; // 4-bit data
int hammingCode[7];
// Generate Hamming Code
calculateHamming(data, hammingCode);
// Print Hamming Code
printf("Hamming Code: ");
for (int i = 0; i < 7; i++) {
printf("%d", hammingCode[i]);
}
printf("\n");
// Simulate a single-bit error in transmission
hammingCode[2] = 0; // Intentionally flipping a bit
// Detect and correct error
int errorPos = checkHamming(hammingCode);
if (errorPos == 0) {
printf("No error detected.\n");
} else {
printf("Error detected at position: %d\n", errorPos);
hammingCode[errorPos - 1] = !hammingCode[errorPos - 1];
```

```
printf("Corrected Hamming Code: ");
for (int i = 0; i < 7; i++) {
printf("%d", hammingCode[i]);
}
printf("\n");
}
return 0;
}
```

Output

```
Hamming Code: 0110011
Error detected at position: 3
Corrected Hamming Code: 0110011
```

Stop and wait Protocol (c++ code)

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <thread>
#include <chrono>
using namespace std;
// Simulates the Stop-and-Wait protocol for flow control
void stopAndWait(int totalPackets) {
for (int packet = 1; packet <= totalPackets; ++packet) {
cout << "Sending packet " << packet << "..." << endl;
// Simulate random acknowledgment loss or delay
int ack = rand() % 2;
// If ack = 1, the acknowledgment is received; else, it's lost.
if (ack == 1) {
// Simulate transmission delay
this_thread::sleep_for(chrono::seconds(1));
cout << "Acknowledgment received for packet " << packet << "." << endl;
} else {
// Simulate lost acknowledgment; retransmit the packet
cout << "Acknowledgment lost for packet " << packet << ". Retransmitting..." << endl;
packet--; // Retransmit the packet
}
// Simulate delay between packet transmissions
this_thread::sleep_for(chrono::seconds(1));
}
}
int main() {
srand(time(0)); // Initialize random seed for acknowledgment loss simulation
int totalPackets;
// Input: Total number of packets to send
cout << "Enter the total number of packets to send: ";
cin >> totalPackets;
// Simulate the Stop-and-Wait protocol
stopAndWait(totalPackets);
return 0;
}
```

Explanation:

stopAndWait function: Simulates the transmission of a packet from sender to receiver.
Uses random numbers to simulate packet acknowledgment (either received or lost).
If the acknowledgment is lost, the packet is retransmitted.

this_thread::sleep_for: Simulates the delay between transmission and acknowledgment.

Sample Input/Output:

Enter the total number of packets to send: 5

Possible Output:

Sending packet 1...
Acknowledgment received for packet 1.
Sending packet 2...
Acknowledgment received for packet 2.
Sending packet 3...
Acknowledgment lost for packet 3. Retransmitting...
Sending packet 3...
Acknowledgment received for packet 3.
Sending packet 4...
Acknowledgment received for packet 4.
Sending packet 5...
Acknowledgment lost for packet 5. Retransmitting...
Sending packet 5...
Acknowledgment received for packet 5.

In this output, the acknowledgment for packet 3 and packet 5 was lost, leading to retransmission of those packets. The behavior may change each time the program is run due to the randomness of acknowledgment loss simulation.

Sliding Window Protocol (Go-Back-N)

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <thread>
#include <chrono>
using namespace std;
void goBackN(int totalPackets, int windowSize) {
int nextPacket = 1;
while (nextPacket <= totalPackets) {
for (int i = 0; i < windowSize && nextPacket + i <= totalPackets; ++i) {
cout << "Sending packet " << nextPacket + i << "..." << endl;
}
for (int i = 0; i < windowSize && nextPacket + i <= totalPackets; ++i) {
int ack = rand() % 2;
if (ack == 1) {
this_thread::sleep_for(chrono::seconds(1));
cout << "Acknowledgment received for packet " << nextPacket + i << "." << endl;
} else {
cout << "Acknowledgment lost for packet " << nextPacket + i << ". Retransmitting window..." << endl;
break;
}
}
nextPacket += windowSize;
}
}
int main() {
srand(time(0)); // Initialize random seed
int totalPackets, windowSize;
cout << "Enter the total number of packets to send: ";
cin >> totalPackets;
cout << "Enter the window size: ";
cin >> windowSize;
```

```
goBackN(totalPackets, windowSize);
return 0;
}
```

Sample Input/Output for Go-Back-N:

Input:

```
Enter the total number of packets to send: 5
Enter the window size: 3
```

Output:

```
Sending packet 1...
Sending packet 2...
Sending packet 3...
Acknowledgment received for packet 1.
Acknowledgment lost for packet 2. Retransmitting window...
Sending packet 2...
Sending packet 3...
Sending packet 4...
Acknowledgment received for packet 2.
Acknowledgment received for packet 3.
Acknowledgment received for packet 4.
```

Sliding Window Protocol (Selective Repeat)

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <thread>
#include <chrono>
using namespace std;
void selectiveRepeat(int totalPackets, int windowSize) {
bool acks[totalPackets + 1] = {false}; // Array to track acknowledgments
int sentPackets = 0;
while (sentPackets < totalPackets) {
for (int i = 0; i < windowSize && sentPackets + i < totalPackets; ++i) {
if (!acks[sentPackets + i]) {
cout << "Sending packet " << sentPackets + i + 1 << "..." << endl;
}
}
for (int i = 0; i < windowSize && sentPackets + i < totalPackets; ++i) {
if (!acks[sentPackets + i]) {
int ack = rand() % 2;
if (ack == 1) {
this_thread::sleep_for(chrono::seconds(1));
cout << "Acknowledgment received for packet " << sentPackets + i + 1 << "." << endl;
acks[sentPackets + i] = true;
} else {
cout << "Acknowledgment lost for packet " << sentPackets + i + 1 << ". Retransmitting..." << endl;
}
}
for (int i = 0; i < totalPackets; ++i) {
if (acks[i]) {
++sentPackets;
}
}
}
}
int main() {
srand(time(0)); // Initialize random seed
```

```
int totalPackets, windowSize;
cout << "Enter the total number of packets to send: ";
cin >> totalPackets;
cout << "Enter the window size: ";
cin >> windowSize;
selectiveRepeat(totalPackets, windowSize);
return 0;
}
```

Input:

```
Enter the total number of packets to send: 5
Enter the window size: 3
```

Output:

```
Sending packet 1...
Sending packet 2...
Sending packet 3...
Acknowledgment received for packet 1.
Acknowledgment lost for packet 2. Retransmitting...
Acknowledgment received for packet 3.
Sending packet 2...
Acknowledgment received for packet 2.
```